Practical: 07 **Date: 01-10-2026**

Addressing underfitting and overfitting through validation curve

Procedure:

Step 1: Load and Preprocess the Dataset

- Import pandas and load the Breast Cancer Wisconsin dataset (wdbc.data) using pd.read_csv().
- Extract the 30 numerical features into array X and class labels ('M'/'B') into array y.
- Convert target labels into integers (1 for Malignant, 0 for Benign) using LabelEncoder.
- Split the dataset into 80% training and 20% testing sets using train_test_split.

Step 2: Build the Machine Learning Pipeline

- Create a Pipeline combining StandardScaler for feature scaling and LogisticRegression as the classifier.

Step 3: Diagnose Bias and Variance with Learning Curves

- Use learning_curve with 10-fold cross-validation (cv=10) across varying training sample sizes (np.linspace(0.1, 1.0, 10)).
- Calculate the mean and standard deviation for training and validation scores.
- Plot training accuracy and validation accuracy along with their standard deviation bands (plt.fill_between).
- Display the learning curve plot to observe model performance relative to the number of training samples.

Step 4: Address Overfitting/Underfitting with Validation Curves

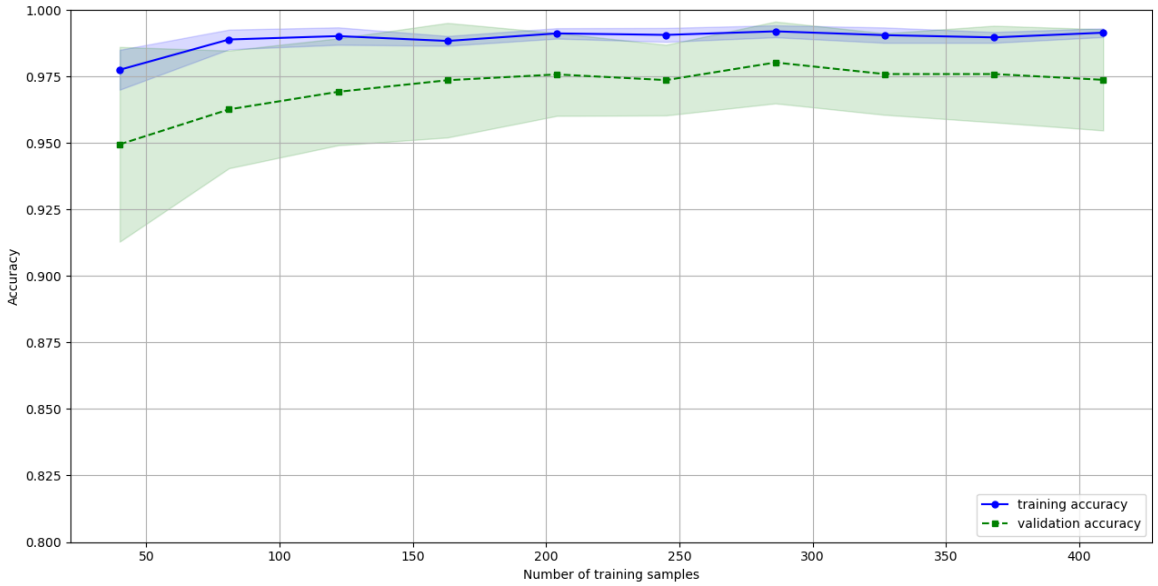
- Define a range of inverse regularization parameter values (param_range = [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]).
- Compute training and validation scores across these hyperparameter values using validation_curve targeting clf__C.
- Calculate the mean and standard deviation for both scores.
- Plot training and validation accuracy against Parameter C on a logarithmic scale (plt.xscale('log')) with standard deviation bands.
- Analyze the validation curve plot to identify the optimal value of parameter C that balances bias and variance.

Solution:

```
import pandas as pd
df = pd.read_csv(r"C:\Users\Kavya Shettigar\Downloads\KAVYA SDM College(Autonomous), Ujire - wdbc.data", header=None)
df.head()
from sklearn.preprocessing import LabelEncoder
X = df.loc[:, 2:].values
y = df.loc[:, 1].values
le = LabelEncoder()
y = le.fit_transform(y)
le.transform(['M', 'B'])
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=1)
```

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import learning_curve
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
pipe_lr = Pipeline([('scl', StandardScaler()),
                    ('clf', LogisticRegression(penalty='l2', random_state=0))])
train_sizes, train_scores, test_scores = learning_curve(estimator=pipe_lr,
                                                        X=X_train, y=y_train,
                                                        train_sizes=np.linspace(0.1, 1.0, 10),
                                                        cv=10,
                                                        n_jobs=1)
train_mean = np.mean(train_scores, axis=1)
train_std = np.std(train_scores, axis=1)
test_mean = np.mean(test_scores, axis=1)
test_std = np.std(test_scores, axis=1)
```

```python
plt.plot(train_sizes, train_mean, color='blue', marker='o', markersize=5, label='training accuracy')
plt.fill_between(train_sizes,
                 train_mean + train_std,
                 train_mean - train_std,
                 alpha=0.15, color='blue')
plt.plot(train_sizes, test_mean, color='green', linestyle='--', marker='s', markersize=5, label='validation accuracy')
plt.fill_between(train_sizes,
                 test_mean + test_std,
                 test_mean - test_std,
                 alpha=0.15, color='green')
plt.grid()
plt.xlabel('Number of training samples')
plt.ylabel('Accuracy')
plt.legend(loc='lower right')
plt.ylim([0.8, 1.0])
plt.show()
```



```
from sklearn.model_selection import validation_curve
param_range = [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]
train_scores, test_scores = validation_curve(estimator=pipe_lr,
                                             X=X_train, y=y_train,
                                             param_name='clf__C',
                                             param_range=param_range,
                                             cv=10)
train_mean = np.mean(train_scores, axis=1)
train_std = np.std(train_scores, axis=1)
test_mean = np.mean(test_scores, axis=1)
test_std = np.std(test_scores, axis=1)
plt.plot(param_range, train_mean, color='blue', marker='o', markersize=5, label='training accuracy')
plt.fill_between(param_range,
                 train_mean + train_std,
                 train_mean - train_std,
                 alpha=0.15, color='blue')
plt.plot(param_range, test_mean, color='green', linestyle='--', marker='s', markersize=5, label='validation accuracy')
plt.fill_between(param_range,
                 test_mean + test_std,
                 test_mean - test_std,
                 alpha=0.15, color='green')
```

```
plt.grid()
plt.xscale('log')
plt.legend(loc='lower right')
plt.xlabel('Parameter C')
plt.ylabel('Accuracy')
plt.ylim([0.8, 1.0])
plt.show()
```

